МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра _____

КУРСОВАЯ РАБОТА

по дисциплине «Разработка серверных частей интернет-ресурсов»

Тема курсовой работы: «Серверная часть системы стриминга контента»

Студент группы ИКБО-21-23	Шлеев М. _____ _____
Руководитель курсовой работы	_____ _____
Работа представлена к защите	«____» _____ 2026 г.
Допущен к защите	«____» _____ 2026 г.

Москва 2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Дисциплина: «Разработка серверных частей интернет-ресурсов».

Студент: Шлеев М., группа ИКБО-21-23.

Тема: «Серверная часть системы стриминга контента».

Исходные данные: требования к оформлению и содержанию — учебно-методическое пособие по выполнению курсовой работы [20] и ГОСТ 7.32-2017; архитектурный паттерн — чистая архитектура; язык программирования — PHP; СУБД — PostgreSQL.

Перечень вопросов, подлежащих разработке:

1. Анализ предметной области: технологии потоковой доставки, сравнение аналогов, функциональные и нефункциональные требования, матрица отслеживаемости.
2. Выбор и обоснование технологий, паттернов проектирования и средств развёртывания.
3. Разработка архитектуры приложения на основе чистой архитектуры.
4. Разработка серверной части интернет-ресурса: база данных, программный интерфейс, фоновая обработка видео, тестирование.

Срок представления к защите: «____» _____ 2026 г.

Задание выдал: _____ / _____ /

Задание получил: _____ / Шлеев М. /

ГЛОССАРИЙ

Адаптер (интерфейсный) — компонент внешнего слоя архитектуры, преобразующий данные между форматом, удобным для сценариев использования, и форматом внешнего средства (HTTP-запросы, строки таблиц базы данных).

Внедрение зависимостей — приём, при котором объект получает необходимые ему зависимости извне (обычно через конструктор), а не создаёт их сам.

Кодек — программа или алгоритм кодирования и декодирования аудио- и видеоданных (например, H.264 для видео, AAC для звука).

Манифест (плейлист) — текстовый файл формата HLS (расширение .m3u8), перечисляющий сегменты медиапотока в порядке воспроизведения.

Очередь задач — хранилище заданий на обработку, из которого отдельный процесс-исполнитель последовательно выбирает и выполняет их.

Подписанная ссылка — ссылка на ресурс, содержащая срок действия и цифровую подпись; сервер принимает запрос только при совпадении подписи и неистёкшем сроке.

Порт — интерфейс, объявленный во внутреннем слое архитектуры и описывающий то, что сценарию использования нужно от внешней среды; реализуется в слое инфраструктуры.

Правило зависимостей — требование чистой архитектуры: исходный код может зависеть только от кода своего или более внутреннего слоя.

Сегмент — короткий фрагмент медиапотока (в работе — около 6 секунд), передаваемый клиенту отдельным HTTP-запросом.

Сценарий использования — класс прикладного слоя, реализующий одно действие пользователя или системы (регистрация, запрос потока и т. п.).

Стриминг (потоковая передача) — способ доставки аудио- и видеоданных, при котором воспроизведение начинается до полной загрузки файла.

Сущность — объект предметной области, обладающий идентичностью и правилами поведения (пользователь, контент, подписка).

Транскодирование — преобразование медиафайла из одного формата, кодека или набора параметров в другой.

Чистая архитектура — способ организации программы в виде концентрических слоёв, в котором бизнес-правила не зависят от фреймворков, базы данных и способа доставки данных.

Middleware (промежуточный обработчик) — компонент, выполняемый до или после основного обработчика запроса (проверка токена, преобразование ошибок).

СПИСОК ПРИНЯТЫХ СОКРАЩЕНИЙ

API — Application Programming Interface — программный интерфейс приложения

CDN — Content Delivery Network — сеть доставки содержимого

DASH — Dynamic Adaptive Streaming over HTTP — динамическая адаптивная потоковая передача по HTTP

DRM — Digital Rights Management — управление цифровыми правами

HLS — HTTP Live Streaming — потоковая передача по протоколу HTTP от компании Apple

HMAC — Hash-based Message Authentication Code — код аутентификации сообщения на основе хеш-функции

HTTP — HyperText Transfer Protocol — протокол передачи гипертекста

HTTPS — HTTP Secure — HTTP поверх шифрованного соединения

JSON — JavaScript Object Notation — текстовый формат обмена данными

JWT — JSON Web Token — токен доступа в формате JSON

PDO — PHP Data Objects — расширение PHP для работы с базами данных

PSR — PHP Standards Recommendation — рекомендация по стандартам PHP

REST — Representational State Transfer — стиль построения сетевых интерфейсов

SHA — Secure Hash Algorithm — семейство криптографических хеш-функций

SQL — Structured Query Language — язык структурированных запросов

TLS — Transport Layer Security — протокол защиты транспортного уровня

TTL — Time To Live — время жизни

UML — Unified Modeling Language — унифицированный язык моделирования

URL — Uniform Resource Locator — единообразный указатель ресурса

БД — база данных

ИС — информационная система

ПО — программное обеспечение

СУБД — система управления базами данных

ГОСТ — государственный стандарт

ISO — International Organization for Standardization — Международная организация по стандартизации

IEC — International Electrotechnical Commission — Международная электротехническая комиссия

RFC — Request for Comments — документ серии стандартов Интернета

СПИСОК ИСПОЛЬЗУЕМЫХ НОРМАТИВНЫХ ДОКУМЕНТОВ

ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления.

ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.

ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов.

ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering.

ISO/IEC 23009-1:2022. Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats.

ISO/IEC 19505-2:2012. Information technology — Object Management Group Unified Modeling Language (OMG UML) — Part 2: Superstructure.

RFC 8216. HTTP Live Streaming. IETF, 2017.

RFC 7519. JSON Web Token (JWT). IETF, 2015.

RFC 2104. HMAC: Keyed-Hashing for Message Authentication. IETF, 1997.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....8
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....9
1.1 Потоковая передача контента и состав серверной части.....9
1.2 Технологии адаптивной потоковой доставки по HTTP.....10
1.3 Анализ аналогов.....11
1.3.1 YouTube.....11
1.3.2 Netflix.....11
1.3.3 Twitch.....12
1.3.4 Кинопоиск.....12
1.4 Требования к системе.....13
1.4.1 Пользователи и варианты использования.....13
1.4.2 Функциональные требования.....13
1.4.3 Нефункциональные требования.....14
1.5 Матрица отслеживаемости.....16
1.6 Выводы по главе 1.....17
2 ВЫБОР И ОБОСНОВАНИЕ ТЕХНОЛОГИЙ.....18
2.1 Выбор архитектурного паттерна.....18
2.2 Выбор языка программирования.....19
2.3 Выбор веб-слоя и контейнера внедрения зависимостей.....20
2.4 Выбор СУБД и организация очереди задач.....21
2.5 Выбор формата доставки видео.....22
2.6 Выбор средств аутентификации и защиты доступа.....22
2.6.1 Хранение паролей.....22
2.6.2 Токены доступа.....23
2.6.3 Подписанные ссылки на видео.....23
2.7 Выбор средств развёртывания и тестирования.....23
2.8 Итоговый технологический стек.....25
2.9 Выводы по главе 2.....25
3 РАЗРАБОТКА АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ НА ОСНОВЕ ВЫБРАННОГО ПАТТЕРНА.....26
3.1 Слои архитектуры и правило зависимостей.....26
3.2 Слой сущностей (Domain).....27
3.3 Слой сценариев использования (Application).....29
3.4 Слой адаптеров интерфейсов (Interfaces).....31

3.5 Слой инфраструктуры (Infrastructure).....33
3.6 Взаимодействие слоёв.....34
3.7 Соответствие архитектуры требованиям и допущенные компромиссы.....36
3.8 Выводы по главе 3.....37
4 РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ИНТЕРНЕТ-РЕСУРСА.....38
4.1 Структура проекта и запуск.....38
4.2 База данных.....38
4.3 Реализация ключевых механизмов.....39
4.3.1 Регистрация, вход и роли.....39
4.3.2 Загрузка видео и очередь транскодирования.....40
4.3.3 Транскодирование в HLS.....40
4.3.4 Выдача видео по подписанным ссылкам.....41
4.3.5 Подписки, позиция просмотра и история.....41
4.3.6 Обработка ошибок и меры защиты.....41
4.4 Тестирование и результаты проверки.....42
4.4.1 Автоматические тесты.....42
4.4.2 Ручное тестирование.....42
4.5 Ограничения и направления развития.....43
4.6 Выводы по главе 4.....44
ЗАКЛЮЧЕНИЕ.....45
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....47
ПРИЛОЖЕНИЕ А Схема базы данных.....50

ВВЕДЕНИЕ

Потоковая передача видео стала основным способом доставки аудиовизуального контента: онлайн-кинотеатры, видеохостинги и образовательные платформы обслуживают одновременно большое число зрителей, и от них требуется стабильная работа при высокой нагрузке. За пользовательским интерфейсом таких сервисов стоит серверная часть веб-приложения [1], которая принимает и подготавливает видеофайлы, управляет каталогом и правами доступа, выдаёт данные клиентам и ведёт учёт просмотров. От качества её проектирования зависят безопасность, расширяемость и стоимость сопровождения сервиса, что делает актуальной разработку серверной части с чётко определённой архитектурой.

Цель работы — разработать серверную часть системы стриминга контента на основе чистой архитектуры, обеспечивающую загрузку и подготовку видео к потоковой доставке, защищённую выдачу видео подписчикам и учёт истории просмотров.

Задачи: проанализировать предметную область и сформулировать требования; выбрать технологии; спроектировать архитектуру по слоям; реализовать серверную часть и проверить её тестами.

Объект исследования — процесс потоковой доставки видеоконтента пользователям интернет-ресурса. Предмет исследования — архитектура и программная реализация серверной части, обеспечивающей этот процесс.

Ожидаемый результат — работающее серверное приложение на PHP с интерфейсом REST, базой данных PostgreSQL, фоновой обработкой видео и автоматическими тестами.

Работа состоит из четырёх глав (анализ предметной области, выбор технологий, архитектура, реализация и тестирование), заключения, списка источников и приложения.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

В настоящей главе рассматривается предметная область систем потокового вещания контента: описываются состав и принципы работы таких систем, технологии адаптивной доставки видео, анализируются популярные платформы-аналоги, формулируются требования к разрабатываемой серверной части и строится матрица отслеживаемости, связывающая требования с реализацией и способами проверки.

1.1 Потоковая передача контента и состав серверной части

Потоковая передача мультимедиа (стриминг) — способ доставки аудио- и видеоданных, при котором воспроизведение начинается до получения файла целиком: данные передаются клиенту частями и воспроизводятся по мере поступления. Различают две основные модели. В модели видео по запросу (video on demand, VOD) контент заранее записан и обработан, а зритель выбирает его из каталога. В модели прямой трансляции (live) контент создаётся в момент просмотра и обрабатывается с минимальной задержкой. Разрабатываемая в курсовой работе система реализует модель VOD.

Серверная часть системы стриминга решает несколько взаимосвязанных задач (общие принципы проектирования и разработки веб-приложений изложены в [1]). Во-первых, она хранит и отдаёт метаданные: каталог произведений, жанры, описания, статус публикации. Во-вторых, принимает исходные видеофайлы и готовит их к доставке: перекодирует и нарезает на короткие сегменты. В-третьих, хранит подготовленные сегменты и плейлисты и доставляет их клиентам по протоколу HTTP. В-четвёртых, управляет доступом: аутентифицирует пользователей, разграничивает роли и проверяет право на просмотр платного контента. Наконец, она учитывает состояние зрителя: подписки, позицию остановки и историю просмотров. Обобщённая схема потока данных показана на рисунке 1.

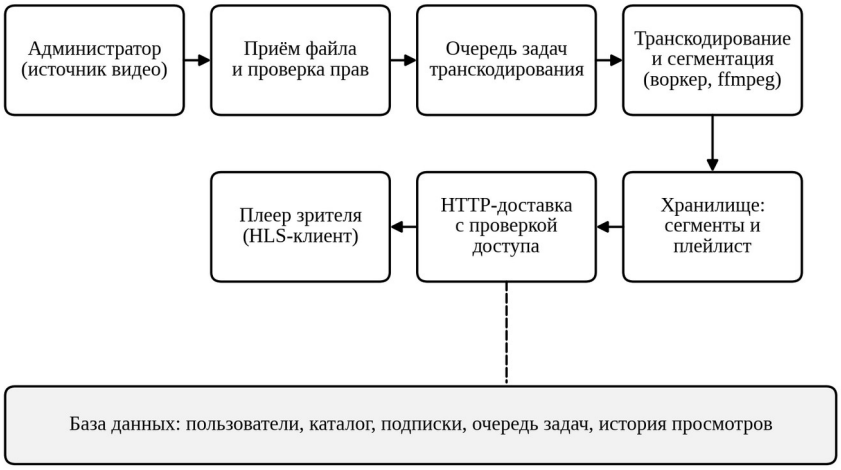


Рисунок 1 – Обобщённая схема потоковой доставки видео по запросу

Качество подобных систем принято оценивать по модели, закреплённой в ГОСТ Р ИСО/МЭК 25010-2015 [2]: она включает восемь характеристик качества программного продукта — функциональную пригодность, производительность, совместимость, удобство использования, надёжность, защищённость, сопровождаемость и переносимость. По Клеппману, для систем с интенсивной обработкой данных наиболее важны надёжность, масштабируемость и сопровождаемость [3]. Эти характеристики используются в разделе 1.4 при формулировке нефункциональных требований.

1.2 Технологии адаптивной потоковой доставки по HTTP

Современные платформы доставляют видео по протоколу HTTP: ролик нарезается на короткие сегменты, а клиент последовательно запрашивает их, как обычные файлы. Как отмечает И. Содагар, такой подход выгоден тем, что серверная инфраструктура HTTP широко распространена и хорошо масштабируется, а управление воспроизведением выполняет сам клиент [4]. Наиболее распространены два семейства технологий: HLS и MPEG-DASH.

Протокол HTTP Live Streaming (HLS) описан в RFC 8216 [5]. Медиаданные делятся на сегменты, а клиенту выдаётся текстовый плейлист формата M3U8, перечисляющий адреса сегментов и их длительности. Главный плейлист (master playlist) может содержать несколько вариантов потока разного качества, между которыми клиент переключается в зависимости от условий сети.

Технология MPEG-DASH стандартизована в ISO/IEC 23009-1 [6]. Стандарт определяет формат описания медиапредставления (MPD) и форматы сегментов, но не регламентирует работу плееров и эвристики выбора качества

[4]. Обзор А. Бентейлеба и соавторов подтверждает, что алгоритмы адаптации битрейта выполняются на стороне клиента, а стандарты не предписывают конкретного алгоритма [7].

Для серверной части из этого следует важный вывод: независимо от выбранного формата сервер выступает хранилищем подготовленных файлов с контролем доступа, а «интеллект» адаптации сосредоточен в плеере. Поэтому задачи серверной части сводятся к подготовке сегментов и плейлистов, управлению доступом и учёту состояния просмотра. Именно они составляют предмет настоящей курсовой работы. Выбор между HLS и DASH обосновывается в главе 2; в рамках реализации формируется один вариант качества, а многоуровневый главный плейлист рассматривается как направление развития.

1.3 Анализ аналогов

Для анализа выбраны четыре платформы, представляющие разные модели: YouTube (видеохостинг с пользовательским контентом), Netflix (сервис по подписке с собственным каталогом), Twitch (платформа прямых трансляций) и Кинопоиск (российский онлайн-кинотеатр). Сведения получены из открытых источников — справочных материалов и технических блогов компаний. Внутренняя реализация этих систем в значительной части не раскрывается, поэтому сравнение проводится по документированным свойствам.

1.3.1 YouTube

YouTube — видеохостинг, в котором источником контента являются сами пользователи. Требования к загружаемым файлам публикуются в справке: рекомендуются контейнер MP4, видеокодек H.264 и аудиокодек AAC-LC [8]. Загруженное видео перекодируется платформой в несколько разрешений, чтобы зритель мог выбрать вариант, подходящий скорости соединения [9]. Наряду с бесплатным просмотром существует платное членство на каналах [10].

1.3.2 Netflix

Netflix — сервис с платной подпиской: тарифы различаются качеством изображения, числом одновременно работающих устройств и возможностью загрузки видео [11]. Каталог кодируется силами самого сервиса. В публикации 2015 года Netflix описал подход per-title: для каждого произведения строится индивидуальная «лестница» битрейтов вместо единой для всего каталога [12].

1.3.3 Twitch

Twitch — платформа прямых трансляций. По данным технического блога компании, серверы приёма потоков транскодируют трансляции в разные битрейты и форматы для сети доставки контента, а для записей формируются плейлисты с версиями разного качества [13]. Подписка на канал — ежемесячный платёж в поддержку стримера, дающий зрителю определённые преимущества [14].

1.3.4 Кинопоиск

Кинопоиск — онлайн-кинотеатр компании «Яндекс». Доступ предоставляется по подписке «Яндекс Плюс», а также через покупку и аренду отдельных фильмов. Подпиской можно поделиться с тремя близкими, у каждого из которых будет свой профиль; предусмотрены детский раздел с возрастным ограничением и скачивание для просмотра без сети [15].

Результаты сравнения сведены в таблицу 1.

Таблица 1 — Сравнение платформ-аналогов

Платформа	**Источник контента**	**Подготовка видео к доставке**	**Модель доступа**
YouTube	Пользователи загружают видео [8]	Перекодирование в несколько разрешений [9]	Бесплатный просмотр; платное членство на каналах [10]
Netflix	Каталог формируется сервисом	Индивидуальная лестница битрейтов для каждого произведения [12]	Подписка с несколькими тарифами [11]
Twitch	Стримеры в прямом эфире; записи трансляций [13]	Транскодирование в разные битрейты и форматы [13]	Подписка на канал с дополнительными преимуществами [14]
Кинопоиск	Каталог фильмов и сериалов [15]	В открытых источниках не раскрывается	Подписка, покупка и аренда [15]
Разрабатываемая система	Администратор загружает видеофайлы	Транскодирование в HLS (один вариант качества) отдельным процессом через очередь задач	Бесплатный и платный контент; подписка на 30 дней по тарифу

Анализ показывает, что у платформ общий набор базовых механизмов: подготовка видео в нескольких вариантах качества, отделение приёма и обработки контента от его доставки, управление доступом на основе аккаунта и подписки. Преимущества рассмотренных решений — масштаб, отработанная адаптивная доставка и зрелые модели монетизации. С точки зрения учебной задачи их недостатками являются закрытость внутреннего

устройства и невозможность воспроизвести промышленную инфраструктуру (сети доставки контента, распределённые точки приёма) в рамках курсовой работы.

Из анализа заимствуются следующие решения: от YouTube — приём исходного файла с проверкой формата и последующая перекодировка; от Netflix и Кинопоиска — подписочная модель доступа к платному контенту с ограничением по времени; от Twitch — вынесение транскодирования в отдельный контур обработки, не зависящий от запросов зрителей [13].

1.4 Требования к системе

1.4.1 Пользователи и варианты использования

В системе выделены два типа пользователей и один системный участник. Зритель регистрируется, входит в систему, просматривает каталог, запрашивает воспроизведение, оформляет подписку, сохраняет позицию просмотра и просматривает историю. Администратор обладает возможностями зрителя и дополнительно создаёт карточки контента и загружает видеофайлы. Системным участником является фоновый обработчик (воркер), который берёт задачи из очереди и выполняет транскодирование; на диаграмме вариантов использования (рисунок 2) он не показан, так как не взаимодействует с пользователями напрямую.



Рисунок 2 – Диаграмма вариантов использования системы

1.4.2 Функциональные требования

Требования сформулированы с учётом рекомендаций ISO/IEC/IEEE 29148 [16]: каждое требование выражено одним утверждением, однозначно и допускает проверку.

ФТ-01. Система должна регистрировать пользователя по адресу электронной почты и паролю длиной не менее 8 символов; адрес приводится к нижнему регистру и должен быть уникальным.

ФТ-02. Система должна аутентифицировать пользователя и выдавать токен доступа с ограниченным сроком действия.

ФТ-03. Система должна разграничивать права по ролям «зритель» и «администратор».

ФТ-04. Система должна выдавать каталог опубликованного контента с поиском по названию, фильтром по жанру и постраничным выводом.

ФТ-05. Система должна позволять администратору создавать карточки контента.

ФТ-06. Система должна принимать от администратора исходный видеофайл допустимого формата и ставить задачу транскодирования в очередь.

ФТ-07. Система должна асинхронно преобразовывать исходное видео в формат HLS и менять статус контента (черновик, обработка, опубликован, ошибка).

ФТ-08. Система должна перед воспроизведением проверять право просмотра (бесплатный контент либо активная подписка) и выдавать ссылку на плейлист с ограниченным сроком действия.

ФТ-09. Система должна выдавать плейлист и сегменты только по действительной подписи ссылки.

ФТ-10. Система должна оформлять подписку на выбранный тариф; продолжительность определяется тарифом, а продление начинается после окончания текущей подписки.

ФТ-11. Система должна сохранять позицию просмотра для пары «пользователь — контент».

ФТ-12. Система должна выдавать пользователю историю просмотров с последними сохранёнными позициями.

1.4.3 Нефункциональные требования

Нефункциональные требования структурированы по характеристикам качества ГОСТ Р ИСО/МЭК 25010-2015 [2] и приведены в таблице 2.

Таблица 2 — Нефункциональные требования

Характеристика качества	**Требование**	**Способ обеспечения**
Функциональная пригодность	Система реализует функции ФТ-01…ФТ-12 в полном объёме	Трассируемость требований и проверка по таблице 3

Характеристика качества	Требование	Способ обеспечения
Производительность	Длительные операции не блокируют обработку запросов зрителей; выдача ссылки на просмотр не зависит от длительности транскодирования	Транскодирование выполняется отдельным процессом (воркером); очередь задач хранится в базе данных, задачи берутся с блокировкой строк (FOR UPDATE SKIP LOCKED)
Совместимость	Выдаваемые ссылки воспроизводятся стандартными HLS-плеерами	Формат HLS по RFC 8216 [5]: плейлист M3U8 и сегменты MPEG-TS; демонстрационная страница использует библиотеку hls.js, браузер Safari воспроизводит HLS самостоятельно
Удобство использования	Ответы API единообразны; ошибки сопровождаются понятным текстом	Ответы в формате JSON; коды 401, 403, 404, 409, 422; единая точка обработки исключений
Надёжность	Состояние каждой задачи транскодирования и каждого контента отслеживается; сбой одной задачи не останавливает обработчик	Статусы задач (pending, running, done, failed) и контента хранятся в базе данных; исключения перехватываются, причина сбоя сохраняется
Защищённость	Пароли не хранятся в открытом виде; доступ к функциям определяется ролью; ссылки на видео ограничены по времени и защищены от подделки; исключены выход за пределы каталога и SQL-инъекции [17, 18]	Хеширование паролей (bcrypt); токены JWT (HS256) со сроком действия [19]; подпись ссылок HMAC-SHA256 с параметрами exp и sig; белый список имён файлов; параметризованные запросы PDO
Сопровождаемость	Бизнес-правила не зависят от фреймворков и базы данных; ключевая логика покрыта автоматическими тестами	Чистая архитектура (четыре слоя); 15 тестов PHPUnit: 12 unit-тестов сценариев с in-memory-реализациями репозиториев и 3 архитектурных теста правила зависимостей
Переносимость	Запуск на любом компьютере с Docker без ручной установки зависимостей	Docker Compose: PHP-FPM, nginx, PostgreSQL, воркер

Требования по защищённости опираются на типичные угрозы для веб-приложений и API: подбор и подмену учётных данных, нарушение авторизации на уровне объектов и функций, внедрение SQL-кода. Токены доступа реализуются по стандарту JSON Web Token.

1.5 Матрица отслеживаемости

Для контроля полноты реализации требования связаны с программными компонентами, точками входа API и способами проверки (таблица 3). Автоматические unit-тесты (классы RegisterUserTest, RequestStreamTest и SubscribeToPlanTest) покрывают регистрацию, проверку права просмотра и оформление подписки; остальные требования проверены сценариями ручного тестирования работающей системы с помощью HTTP-запросов.

Таблица 3 — Матрица отслеживаемости функциональных требований

Требование	**Компонент (сценарий)**	**Точка входа**	**Проверка**
ФТ-01	RegisterUser	POST /auth/register	Unit-тесты (4 теста)
ФТ-02	LoginUser, JwtTokenService	POST /auth/login	Ручной сценарий: вход, получение токена
ФТ-03	AuthMiddleware	Защищённые маршруты	Ручной сценарий: отказ 403 при создании контента зрителем, 401 без токена
ФТ-04	ListContent	GET /content	Ручной сценарий: поиск по названию, фильтр по жанру
ФТ-05	CreateContent	POST /content	Ручной сценарий: создание карточки, статус draft
ФТ-06	AttachMedia	POST /content/{id}/media	Ручной сценарий: загрузка файла, ответ 202
ФТ-07	ProcessNextTranscodeJob, FfmpegTranscoder	Воркер (CLI)	Ручной сценарий: плейлист из четырёх сегментов, статус published
ФТ-08	RequestStream	POST /content/{id}/stream	Unit-тесты (5 тестов); ручной сценарий
ФТ-09	StreamController, HmacUrlSigner	GET /stream/{id}/{file}	Ручной сценарий: валидная подпись — 200, подделка — отказ
ФТ-10	SubscribeToPlan	POST /subscriptions	Unit-тесты (3 теста); ручной сценарий
ФТ-11	SaveProgress	PUT /content/{id}/progress	Ручной сценарий: ответ 204, позиция в истории

Требование	Компонент (сценарий)	Точка входа	Проверка
ФТ-12	GetWatchHistory	GET /me/history	Ручной сценарий: выдача сохранённой позиции

1.6 Выводы по главе 1

1. Серверная часть системы стриминга решает задачи хранения метаданных, подготовки сегментов видео, контроля доступа и учёта состояния просмотра. Технологии HLS и MPEG-DASH переносят управление адаптацией на клиента, а сервер выступает хранилищем подготовленных файлов с контролем доступа.

2. Анализ платформ YouTube, Netflix, Twitch и Кинопоиск выявил общие механизмы: несколько вариантов качества, отделение обработки от доставки, подписочный доступ. Они положены в основу разрабатываемой системы.

3. Сформулированы 12 функциональных требований и требования по восьми характеристикам качества; построена матрица отслеживаемости, связывающая требования с реализацией и способами проверки.

4. Результаты главы являются исходными данными для главы 2 — выбора и обоснования технологий и архитектурного паттерна.

2 ВЫБОР И ОБОСНОВАНИЕ ТЕХНОЛОГИЙ

В настоящей главе обосновывается выбор архитектурного паттерна и технологий для серверной части системы стриминга контента. Исходными данными служат требования, сформулированные в главе 1 (таблицы 2 и 3). Для каждого направления — архитектура, язык программирования, веб-слой, хранение данных и очередь задач, формат доставки видео, аутентификация и защита доступа, инфраструктура — рассматриваются альтернативы, приводятся критерии выбора и фиксируются ограничения принятых решений.

2.1 Выбор архитектурного паттерна

Методические указания называют три наиболее распространённых паттерна проектирования архитектуры серверной части: MVC, DDD и чистую архитектуру [20]. Критерии выбора выведены из нефункциональных требований (таблица 2): К1 — изоляция бизнес-правил от базы данных и фреймворка; К2 — возможность тестировать сценарии без инфраструктуры; К3 — соразмерность подхода сложности предметной области; К4 — трудоёмкость освоения и описания.

Паттерн MVC разделяет прикладной код на модель, представление и контроллер; в классификации М. Фаулера это паттерн уровня представления веб-приложения [21]. Для серверной части, отдающей данные в формате JSON, роль представления сводится к сериализации ответа, а сам паттерн не определяет, где размещать бизнес-правила и как изолировать их от хранилища данных. Поэтому MVC решает лишь часть задач и не отвечает на главный вопрос сопровождаемости.

Предметно-ориентированное проектирование (DDD) предлагает строить программу вокруг модели предметной области: единый язык, сущности, объекты-значения, агрегаты, репозитории [22]. Подход оправдан для сложных областей с большим числом взаимосвязанных правил. В разрабатываемой системе правил немного (доступ к платному контенту, срок подписки, проверка входных данных), поэтому полноценное применение DDD с ограниченными контекстами и событиями предметной области было бы избыточным. Вместе с тем отдельные тактические элементы — сущности с собственными правилами и репозитории, объявленные как интерфейсы доменного слоя, — применимы и полезны; их сочетание с разработкой через тестирование и принципом портов и адаптеров рассматривается, например, в [23].

Чистая архитектура Р. Мартина организует систему в концентрические слои: сущности, сценарии использования, адаптеры интерфейсов и внешние детали (базы данных, веб-фреймворки). Ключевое правило — правило

зависимостей: исходный код зависит только от внутренних слоёв, а внутренние слои ничего не знают о внешних [24]. Следствие этого правила — бизнес-правила и сценарии можно тестировать без базы данных и веб-сервера, а базу данных, транскодер или способ выдачи видео — заменять, не затрагивая логику. Именно с этим свойством связаны критерии К1 и К2. Результаты сравнения сведены в таблицу 4.

Таблица 4 — Сравнение архитектурных паттернов

Критерий	**MVC**	**DDD**	**Чистая архитектура**
К1. Изоляция бизнес-правил от БД и фреймворка	Не регламентируется паттерном	Достигается через доменный слой и репозитории	Обеспечивается правилом зависимостей
К2. Тестирование сценариев без инфраструктуры	Зависит от реализации	Высокая	Высокая: порты заменяются тестовыми реализациями
К3. Соразмерность сложности задачи	Подходит, но не решает главных задач	Избыточен при малом числе правил	Соразмерна
К4. Трудоёмкость освоения и описания	Низкая	Высокая (контексты, агрегаты, события)	Средняя

Оценки в таблице носят экспертный характер и основаны на описании паттернов в [21, 22, 24].

Принято решение: использовать чистую архитектуру с отдельными тактическими элементами DDD. Паттерн MVC применяется в сокращённом виде: HTTP-контроллеры размещаются во внешнем слое адаптеров, а представлением служит JSON-ответ. Методические указания отмечают, что полное применение MVC в слое интерфейсов для небольших систем избыточно [20]. Следуя примеру из методических указаний [20], архитектура строится из четырёх слоёв: сущности (Domain), сценарии использования (Application), адаптеры интерфейсов (Interfaces) и инфраструктура (Infrastructure). Подробное описание слоёв приводится в главе 3.

2.2 Выбор языка программирования

Методические указания не ограничивают выбор платформы: в списке рекомендованной литературы представлены книги по PHP, Python и Spring [20]. Для чистой архитектуры язык должен позволять выразить границы слоёв средствами самого языка — интерфейсы, строгую типизацию, неизменяемые объекты — и поддерживать внедрение зависимостей. Сравнение трёх кандидатов приведено в таблице 5.

Таблица 5 — Сравнение платформ для серверной части

Платформа	**Достоинства для решаемой задачи**	**Ограничения**
PHP 8.x	Интерфейсы, строгая типизация (declare(strict_types=1)), свойства только для чтения [25]; каждый запрос обрабатывается независимо; простой запуск (PHP-FPM и nginx); литература в списке методических указаний	Не предназначен для ресурсоёмкой обработки видео — нужен внешний процесс; зависимости и контейнер внедрения подключаются отдельными пакетами
Python (FastAPI, Django)	Лаконичный синтаксис; готовые примеры чистой архитектуры и DDD [23]; богатая экосистема библиотек	Аннотации типов не проверяются интерпретатором, для контроля нужны внешние инструменты
Java (Spring)	Строгая типизация; зрелая экосистема; внедрение зависимостей «из коробки»; пример из методических указаний выполнен на Java EE [20]	Многословность; более сложное окружение сборки и запуска для небольшой учебной системы

Выбран PHP 8.3. Решающими оказались три обстоятельства. Во-первых, возможностей современного PHP достаточно для выражения границ слоёв: в проекте использованы строгая типизация, интерфейсы портов и репозиториев, неизменяемые сущности. Во-вторых, запуск не требует сложного окружения: приложение работает за nginx под управлением PHP-FPM. В-третьих, выбранная ветка поддерживается: по данным php.net, ветка 8.3 получает исправления безопасности до 31 декабря 2027 года, а ветки 8.4 и 8.5 находятся на активной поддержке [26]. Переход на них не потребует изменения архитектуры.

Ограничение выбора связано с тем, что PHP не предназначен для тяжёлых вычислений над видео. Это учтено в архитектуре сознательно: транскодирование выполняется внешней программой ffmpeg, а сценарии работают с ней через порт Transcoder; сама программа запускается отдельным процессом-воркером, а не в обработчике HTTP-запроса.

2.3 Выбор веб-слоя и контейнера внедрения зависимостей

Для приёма HTTP-запросов нужна минимальная прослойка, не навязывающая структуру бизнес-логики. Полноценные фреймворки (Laravel, Symfony) включают собственные средства работы с данными, шаблонизацию и соглашения о структуре каталогов. При чистой архитектуре фреймворк должен находиться во внешнем слое, поэтому большая часть его возможностей осталась бы невостребованной. Выбран микрофреймворк Slim 4, предназначенный для создания простых приложений и API [27]. Slim построен на стандартах PSR-7 (интерфейсы HTTP-сообщений) [28] и PSR-15

(обработчики запросов и промежуточные слои) [29] и не включает собственной реализации PSR-7, поэтому используется пакет slim/psr7.

Это даёт два практических преимущества. Во-первых, контроллеры слоя Interfaces зависят только от интерфейсов PSR-7 и PSR-17, а не от классов Slim; в частности, отдача сегментов выполняется через интерфейс фабрики потоков. Во-вторых, промежуточные слои аутентификации и обработки ошибок реализуют интерфейс PSR-15 и могут быть перенесены в другой совместимый фреймворк. Порядок их выполнения определяется последовательностью добавления: слой обработки ошибок добавляется последним и потому выполняется первым, перехватывая исключения всех остальных слоёв [27].

Внедрение зависимостей выполняет контейнер PHP-DI 7. Все соответствия между интерфейсами портов и их реализациями заданы в одном месте — файле config/container.php (корень композиции). Благодаря этому внутренние слои не содержат ни одного обращения к контейнеру или фреймворку.

2.4 Выбор СУБД и организация очереди задач

Данные системы (пользователи, каталог, тарифы, подписки, задачи транскодирования, история просмотров) имеют чёткую структуру и связи, а часть правил выражается ограничениями целостности: уникальность адреса электронной почты, внешние ключи, допустимые значения статусов. Поэтому выбрана реляционная СУБД PostgreSQL 16. В схеме применяются ограничения CHECK и внешние ключи, а запросы используют конструкции INSERT … RETURNING и INSERT … ON CONFLICT: сохранение позиции просмотра выполняется одним запросом-upsert [30].

Выбор конкретной СУБД влияет только на реализацию репозиториев слоя Interfaces: сценарии и сущности работают с интерфейсами хранилищ. Замена PostgreSQL на другую реляционную СУБД потребовала бы изменить SQL в этих классах, но не затронула бы внутренние слои.

Транскодирование видео — длительная операция, которую нельзя выполнять в цикле обработки HTTP-запроса. Рассматривались три варианта организации очереди: таблица в базе данных, Redis и брокер сообщений (например, RabbitMQ). Выбрана таблица transcode_jobs: она не требует дополнительных сервисов (проще развёртывание, что соответствует требованию переносимости), а состояние задач хранится рядом с остальными данными. Конкурентный доступ нескольких воркеров обеспечивается конструкцией SELECT … FOR UPDATE SKIP LOCKED: строки, уже заблокированные другим обработчиком, пропускаются. Документация PostgreSQL отдельно отмечает, что такой режим не предназначен для общих

задач, но подходит для устранения конфликтов блокировок при работе нескольких потребителей с таблицей-очередью [31].

У выбранного решения есть ограничения. Если воркер аварийно остановится во время обработки, задача останется в статусе running и сама в очередь не вернётся; для устранения этого нужен механизм тайм-аутов или периодического возврата «зависших» задач. При росте нагрузки очередь целесообразно заменить брокером сообщений; благодаря интерфейсу TranscodeJobRepository изменения затронут в основном реализацию репозитория.

2.5 Выбор формата доставки видео

Выбор формата определяется требованиями ФТ-07…ФТ-09 и характеристикой совместимости (таблица 2). Сравнивались три подхода: прогрессивная загрузка единого файла MP4, MPEG-DASH и HLS. При прогрессивной загрузке файл воспроизводится по HTTP-запросам диапазонов, но разбиение на сегменты и переключение качества не предусмотрены. DASH стандартизован в ISO/IEC 23009-1 и описывается манифестом MPD [6]; для него потребовались бы формирование MPD и отдельный клиент в браузере. HLS описан в RFC 8216 [5], использует простой текстовый плейлист и воспроизводится браузером Safari нативно, а в остальных браузерах — библиотекой hls.js, которая реализует клиент HLS поверх HTML5-видео и Media Source Extensions [32].

Выбран HLS. Сегментацию и формирование плейлиста выполняет ffmpeg: его мультиплексор hls создаёт плейлист и сегменты [33]. В реализации использованы видеокодек H.264 (профиль main), аудиокодек AAC (128 кбит/с), пиксельный формат yuv420p и сегменты длительностью 6 секунд с принудительными ключевыми кадрами на их границах. На тестовом ролике длительностью 20 секунд получены четыре сегмента (6, 6, 6 и 2 секунды), что подтверждает корректность выбранных параметров.

Ограничения: формируется один вариант качества, поэтому адаптивное переключение битрейта не используется. Создание многоуровневого главного плейлиста рассматривается как направление развития (глава 4). Файл плейлиста назван master.m3u8 для единообразия интерфейса, хотя по содержанию является медиа-плейлистом.

2.6 Выбор средств аутентификации и защиты доступа

2.6.1 Хранение паролей

Пароли хранятся в виде хеша bcrypt, формируемого функцией password_hash [34]. OWASP рекомендует использовать для хранения паролей

современные алгоритмы с настраиваемой стоимостью вычисления [35]; bcrypt к таким алгоритмам относится, а переход на Argon2id, также поддерживаемый функцией password_hash, рассматривается как возможное улучшение. Следует учитывать особенность bcrypt: учитываются только первые 72 байта пароля.

2.6.2 Токены доступа

Для аутентификации запросов выбраны токены JWT [19] с подписью HS256. Токен самодостаточен, поэтому проверка выполняется без обращения к базе данных и без хранения сессий на сервере. Недостаток подхода — сложность досрочного отзыва токена; он компенсируется коротким сроком действия (один час). Использована библиотека firebase/php-jwt. В процессе работы менеджер пакетов Composer отказался устанавливать её ветку 6.x из-за действующего уведомления безопасности (PKSA-y2cr-5h3j-g3ys), поэтому применена версия 7.0.0, для которой уведомлений безопасности не обнаружено. Секрет подписи рекомендуется делать не короче 32 байт.

2.6.3 Подписанные ссылки на видео

Доступ к плейлисту и сегментам ограничивается подписанными ссылками. Подпись формируется как код аутентификации HMAC на основе SHA-256 [36] от строки, включающей идентификатор контента и момент истечения срока; сравнение подписей выполняется функцией hash_equals, время работы которой не зависит от содержимого. Ссылка выдаётся только после проверки права просмотра (ФТ-08), а плеер запрашивает плейлист и сегменты без заголовков авторизации. Это существенно для нативных HLS-плееров, где управлять заголовками запросов к сегментам не всегда возможно. Для этого сервер при выдаче плейлиста дописывает параметры подписи ко всем строкам с адресами сегментов.

Ограничение подхода: ссылка не привязана к конкретному пользователю и действует на весь контент до истечения срока (по умолчанию 300 секунд), поэтому её передача третьему лицу даёт ему доступ на это время. Системы защиты контента класса DRM в работе не реализуются.

2.7 Выбор средств развёртывания и тестирования

Для воспроизводимого запуска на любом компьютере система описана средствами Docker Compose [37]. Развёртывание состоит из четырёх сервисов (рисунок 3): веб-сервера nginx, приложения на PHP-FPM, базы данных PostgreSQL и воркера транскодирования; ffmpeg установлен в образ PHP. Версии PHP-пакетов фиксируются файлом composer.lock.

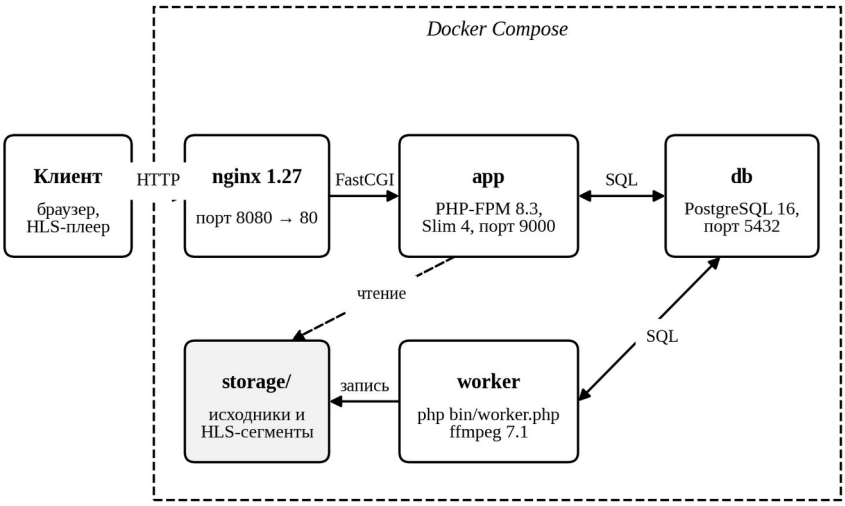


Рисунок 3 – Схема развёртывания системы в Docker Compose

2.8 Итоговый технологический стек

Принятые решения сведены в таблицу 6. Версии указаны по состоянию на установку проекта (04.10.2026) и зафиксированы файлом composer.lock и тегами образов Docker.

Таблица 6 — Технологический стек серверной части

Назначение	**Технология**	**Версия**	**Обоснование**
Язык	PHP	8.3 (в контейнере 8.3.35)	Типизация, интерфейсы, неизменяемые свойства; исправления безопасности до 31.12.2027
Веб-слой	Slim; slim/psr7	4.15.3; 1.8.0	Микрофреймворк на стандартах PSR-7 и PSR-15
Внедрение зависимостей	PHP-DI	7.1.1	Единый корень композиции
Аутентификация	firebase/php-jwt	7.0.0	Токены JWT (RFC 7519), подпись HS256
СУБД и очередь задач	PostgreSQL	16	Реляционная модель, RETURNING, ON CONFLICT, SKIP LOCKED
Транскодирование	ffmpeg	7.1.5	Формирование HLS-плейлиста и сегментов
Формат доставки	HLS	RFC 8216	Нативно в Safari, через hls.js в остальных браузерах
Демонстрационный плеер	hls.js	1.x	Клиент HLS на основе Media Source Extensions
Веб-сервер	nginx; PHP-FPM	1.27; 8.3	Приём запросов и передача их приложению
Тестирование	PHPUnit	10.5.65	Unit-тесты сценариев
Развёртывание	Docker Compose	—	Воспроизводимый запуск четырёх сервисов

2.9 Выводы по главе 2

1. Для разработки серверной части системы стриминга контента выбрана чистая архитектура с отдельными тактическими элементами DDD; паттерн MVC применяется в сокращённом виде во внешнем слое адаптеров.

2. Выбраны PHP 8.3, микрофреймворк Slim 4 с контейнером PHP-DI, PostgreSQL 16 для хранения данных и очереди задач, формат HLS с сегментацией средствами ffmpeg, токены JWT и подписанные ссылки на основе HMAC-SHA256, развёртывание в Docker Compose.

3 РАЗРАБОТКА АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ НА ОСНОВЕ ВЫБРАННОГО ПАТТЕРНА

Для разработки архитектуры выявлены бизнес-правила, на которых строится система: доступ к платному контенту только при активной подписке, срок действия подписки, допустимые состояния контента и задач транскодирования, ограничения на входные данные. Варианты использования системы показаны на рисунке 2 (глава 1). В настоящей главе в соответствии с выбранным в главе 2 паттерном описаны слои архитектуры, их состав и взаимодействие. Диаграммы построены в нотации UML [39, 40] по исходному коду системы, поэтому состав классов и их связи соответствуют реализации.

3.1 Слои архитектуры и правило зависимостей

Чистая архитектура предполагает разделение системы на слои, подчиняющиеся правилу зависимостей: зависимости в исходном коде направлены только от внешних слоёв к внутренним [24]. Применяя этот паттерн, система разделена на четыре слоя. Первый слой — сущности (Domain): бизнес-объекты, правила, которые не зависят ни от приложения, ни от способа хранения данных, и интерфейсы хранилищ. Второй — сценарии использования (Application): действия пользователей над бизнес-объектами и порты — интерфейсы к внешним сервисам, которые нужны сценариям. Третий — адаптеры интерфейсов (Interfaces): HTTP-контроллеры и реализации хранилищ на основе PDO, то есть связь между бизнес-логикой и внешним миром. Четвёртый — инфраструктура (Infrastructure): реализации портов, промежуточные слои HTTP, подключение к базе данных; здесь же подключаются Slim, PHP-DI, библиотека JWT и вызов ffmpeg. Слои показаны на рисунке 4, их состав и допустимые зависимости — в таблице 7.

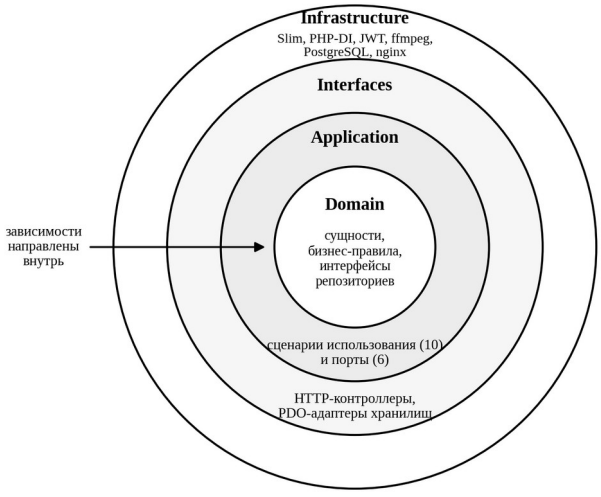


Рисунок 4 – Слои архитектуры системы

Таблица 7 — Слои архитектуры и допустимые зависимости

Слой	**Состав**	**Может зависеть от**	**Не должен зависеть от**
Domain	6 сущностей, 6 исключений, 6 интерфейсов репозиториев (18 классов и интерфейсов)	Только от возможностей языка PHP	Остальных слоёв, фреймворков, PDO
Application	10 сценариев использования и 6 портов (16 классов и интерфейсов)	Domain	Interfaces, Infrastructure, фреймворков (Slim, PSR, JWT, DI), PDO
Interfaces	5 контроллеров (включая базовый), 6 PDO-репозиториев, вспомогательный класс Sql (12 классов)	Domain, Application; интерфейсы PSR-7 и PSR-17; PDO	Infrastructure, Slim, JWT, DI
Infrastructure	2 промежуточных слоя HTTP, 4 класса безопасности, 2 класса работы с медиа, фабрика подключения (9 классов)	Любых внутренних слоёв; Slim, PHP-DI, firebase/php-jwt	—

Правило зависимостей проверяется автоматически. В набор тестов включены три архитектурных теста (класс ArchitectureTest): они просматривают исходный код и убеждаются, что слой не обращается к более внешнему слою, что пространства имён совпадают с каталогами, а слои Domain и Application не используют фреймворки, интерфейсы PSR и PDO. Работоспособность проверки подтверждена экспериментом: при намеренном добавлении в сущность обращения к инфраструктуре и в сценарий обращения к Slim тесты обнаруживали нарушения, после отката изменений все 15 тестов проходят. В текущей реализации нарушений правила зависимостей нет.

3.2 Слой сущностей (Domain)

В слое Domain выделено шесть бизнес-объектов: пользователь (User), тариф (Plan), подписка (Subscription), контент (Content), задача транскодирования (TranscodeJob) и позиция просмотра (WatchProgress). Сущности неизменяемы: поля объявлены только для чтения, а изменение состояния выполняется методами, возвращающими новый экземпляр (например, Content::markPublished). Для каждой сущности объявлен интерфейс репозитория, описывающий потребность предметной области в хранении, но не способ хранения. Диаграмма классов слоя показана на рисунке 5.

Часть бизнес-правил реализована непосредственно в сущностях, поскольку зависит только от самих бизнес-объектов (таблица 8). Остальные

правила, требующие обращения к хранилищу (например, уникальность адреса электронной почты), реализованы в сценариях.

Таблица 8 — Бизнес-правила в слое сущностей

Правило	**Реализация**	**Требование**	**Проверка**
Платный контент доступен только при активной подписке; бесплатный — всем	Content::isAccessibleWith	ФТ-08	Unit-тесты RequestStreamTest
Подписка активна на полуинтервале [начало, окончание)	Subscription::isActiveAt	ФТ-08	Unit-тесты RequestStreamTest (истёкшая подписка)
Окончание подписки равно началу плюс длительность тарифа	Subscription::forPlan	ФТ-10	Unit-тесты SubscribeToPlanTest
Название контента не пусто; год выпуска от 1888 до 2100	Конструктор Content	ФТ-05	Ручной сценарий
Позиция просмотра не отрицательна	Конструктор WatchProgress	ФТ-11	Ручной сценарий
Допустимые статусы контента: draft, processing, published, failed	Content::markProcessing, markPublished, markFailed	ФТ-07	Ручной сценарий

Для сообщения о нарушении правил в слое определены исключения предметной области, унаследованные от DomainException (таблица 9). Каждое исключение несёт рекомендуемый код HTTP-ответа, который использует только внешний слой при формировании ответа. Это сознательный компромисс: строго говоря, знание о протоколе HTTP не относится к предметной области, но такое решение избавляет от отдельной таблицы соответствия «исключение — код» (см. раздел 3.7).

Таблица 9 — Исключения предметной области

Исключение	**Код**	**Когда возникает**
ValidationException	422	Некорректный адрес или короткий пароль, пустое название, неверный год, недопустимое расширение файла, отрицательная позиция просмотра
AuthenticationException	401	Неверные учётные данные, отсутствующий, недействительный или просроченный токен
AccessDeniedException	403	Нет активной подписки, недействительная или истёкшая ссылка, недостаточно прав (не администратор)
NotFoundException	404	Контент или тариф не найден, контент ещё не опубликован

Исключение	Код	Когда возникает
ConflictException	409	Адрес электронной почты уже зарегистрирован

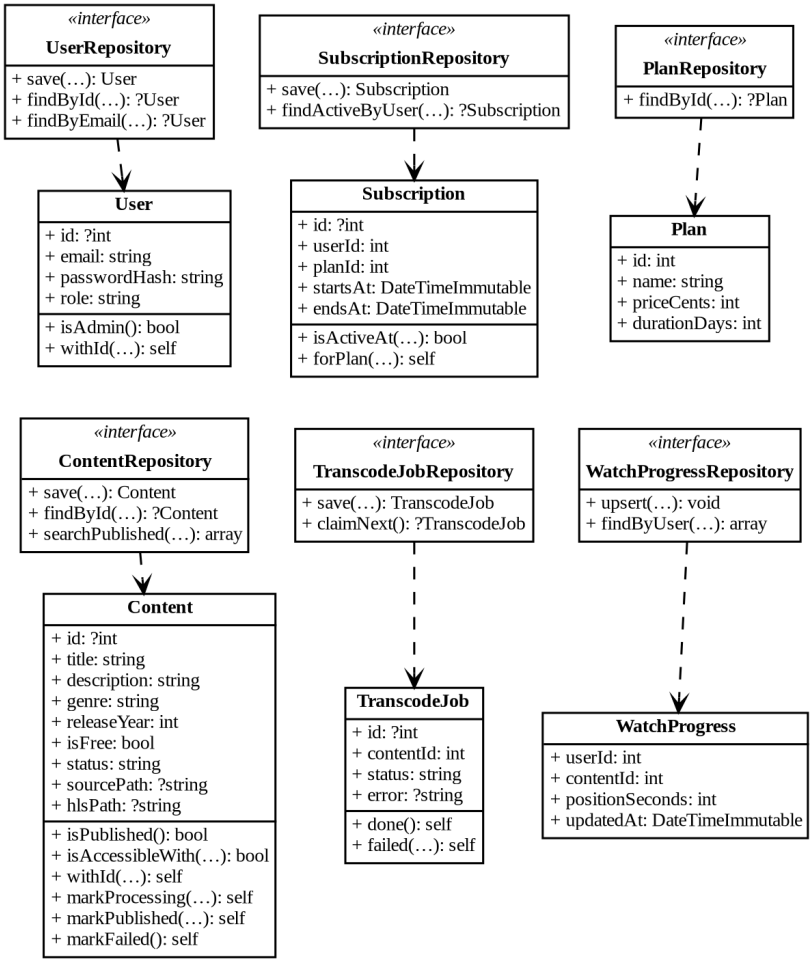


Рисунок 5 – Диаграмма классов слоя сущностей (Domain)

3.3 Слой сценариев использования (Application)

Слой сценариев содержит класс на каждое действие пользователя или системы. Каждый сценарий имеет метод execute, принимает простые типы данных, а не HTTP-запрос, и возвращает сущности или массивы, поэтому не знает ни о протоколе, ни о фреймворке. Зависимости передаются через конструктор; в качестве типов указаны интерфейсы репозиториев из слоя Domain и порты слоя Application. Диаграмма классов сценариев показана на рисунке 6: зависимости изображены как закрытые атрибуты, а основной открытый метод execute — как операция.

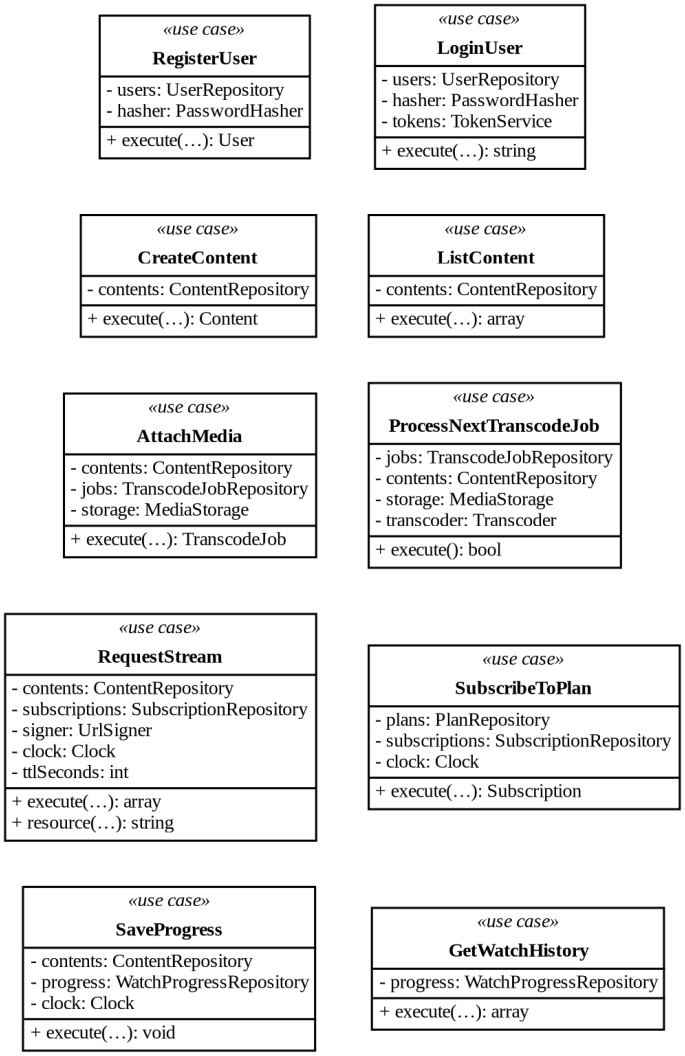


Рисунок 6 – Диаграмма классов слоя сценариев (Application)

Десять сценариев реализуют основные функциональные требования (таблица 3); разграничение ролей (ФТ-03) и выдача файлов по подписи (ФТ-09) реализованы на внешних слоях. Сценарий ProcessNextTranscodeJob вызывается не из HTTP-запроса, а фоновым процессом: он берёт задачу из очереди, запускает транскодирование через порт Transcoder и обновляет статусы контента и задачи. Сценарий RequestStream содержит ключевое правило доступа: проверяет, что контент опубликован, находит активную подписку пользователя на текущий момент и только после этого запрашивает у порта UrlSigner подписанную ссылку. Время всегда берётся из порта Clock, что позволяет в тестах задавать любой момент.

Порты — интерфейсы, через которые сценарии обращаются к внешним сервисам, не зная об их реализации (таблица 10). Порты принадлежат слою Application, а их реализации находятся во внешнем слое Infrastructure, благодаря чему зависимость направлена внутрь.

Таблица 10 — Порты слоя Application

Порт	Методы	Реализация	Назначение
PasswordHasher	hash, verify	BcryptPasswordHasher	Хеширование и проверка паролей
TokenService	issue, verify	JwtTokenService	Выдача и проверка токенов доступа
UrlSigner	sign, verify	HmacUrlSigner	Подпись ссылок на видео
MediaStorage	saveSource, sourceAbsolutePath, hlsDirectory, hlsRelativeDirectory, hlsFile	LocalMediaStorage	Файловое хранилище исходников и HLS-выдачи
Transcoder	toHls	FfmpegTranscoder	Преобразование видео в HLS
Clock	now	SystemClock	Текущее время

3.4 Слой адаптеров интерфейсов (Interfaces)

Слой адаптеров связывает бизнес-логику с внешним миром и состоит из двух групп классов (рисунок 7). Первая группа — HTTP-контроллеры: AuthController, ContentController, StreamController и UserController, унаследованные от BaseController, который содержит общие методы формирования JSON-ответа и чтения тела запроса. Контроллер извлекает данные из запроса, вызывает сценарий и преобразует результат в ответ; бизнес-логики в нём нет. Контроллеры зависят только от интерфейсов PSR-7 и PSR-17. Вторая группа — адаптеры хранилищ: шесть классов Pdo*Repository реализуют интерфейсы репозиториев из слоя Domain и содержат SQL-запросы; именно здесь данные строк таблиц преобразуются в сущности.

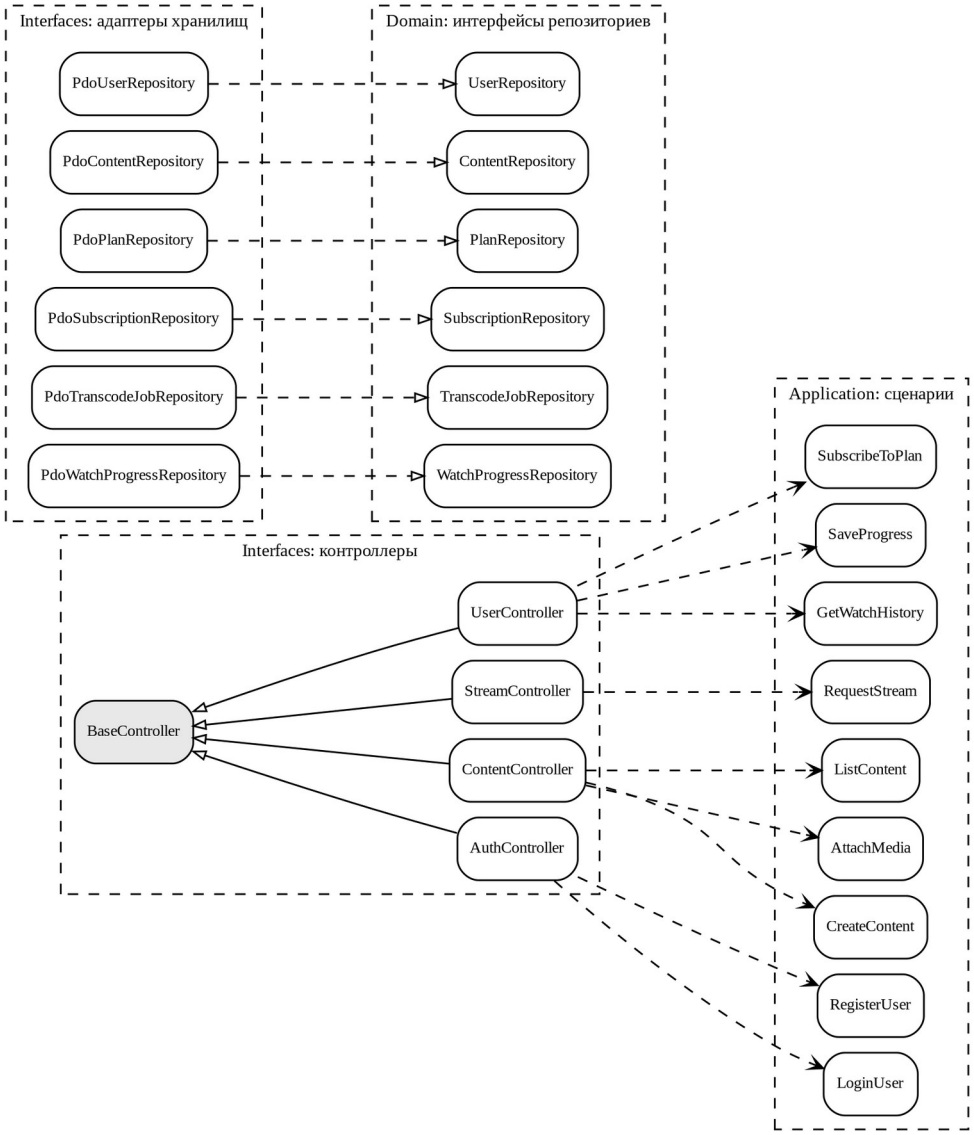


Рисунок 7 – Диаграмма классов слоя адаптеров интерфейсов (Interfaces)

Соответствие маршрутов, контроллеров и сценариев приведено в таблице 11. Маршруты разделены на три группы по способу доступа: публичные, по подписанной ссылке и защищённые токеном; последние делятся на доступные пользователю и только администратору.

Таблица 11 — Маршруты HTTP, контроллеры и сценарии

Метод и путь	**Доступ**	**Контроллер и метод**	**Сценарий**
POST /auth/register	Все	AuthController register	RegisterUser
POST /auth/login	Все	AuthController login	LoginUser
GET /content	Все	ContentController list	ListContent

Метод и путь	Доступ	Контроллер и метод	Сценарий
GET /stream/{id}/{file}	По подписи ссылки	StreamController serve	Порты UrlSigner и MediaStorage (без сценария)
POST /content/{id}/stream	Пользователь (JWT)	StreamController request	RequestStream
POST /subscriptions	Пользователь (JWT)	UserController subscribe	SubscribeToPlan
PUT /content/{id}/progress	Пользователь (JWT)	UserController saveProgress	SaveProgress
GET /me/history	Пользователь (JWT)	UserController history	GetWatchHistory
POST /content	Администратор	ContentController create	CreateContent
POST /content/{id}/media	Администратор	ContentController uploadMedia	AttachMedia

Метод StreamController::serve выдаёт плейлист и сегменты по подписи ссылки без отдельного сценария: операция сводится к проверке подписи и чтению файла. Он также дописывает параметры подписи к адресам сегментов в плейлисте. Это отступление от схемы «одно действие — один сценарий», которое отмечено в разделе 3.7.

3.5 Слой инфраструктуры (Infrastructure)

Слой инфраструктуры содержит то, что нельзя привязать ни к функциям приложения, ни к бизнес-правилам: реализации портов, промежуточные слои HTTP и подключение к базе данных (рисунок 8). Шесть классов реализуют порты слоя Application: BcryptPasswordHasher, JwtTokenService, HmacUrlSigner, LocalMediaStorage, FfmpegTranscoder и SystemClock. Два промежуточных слоя реализуют интерфейс PSR-15: AuthMiddleware проверяет токен и роль, а JsonErrorMiddleware преобразует исключения в JSON-ответы с соответствующим кодом. Класс ConnectionFactory создаёт подключение PDO по параметрам окружения.

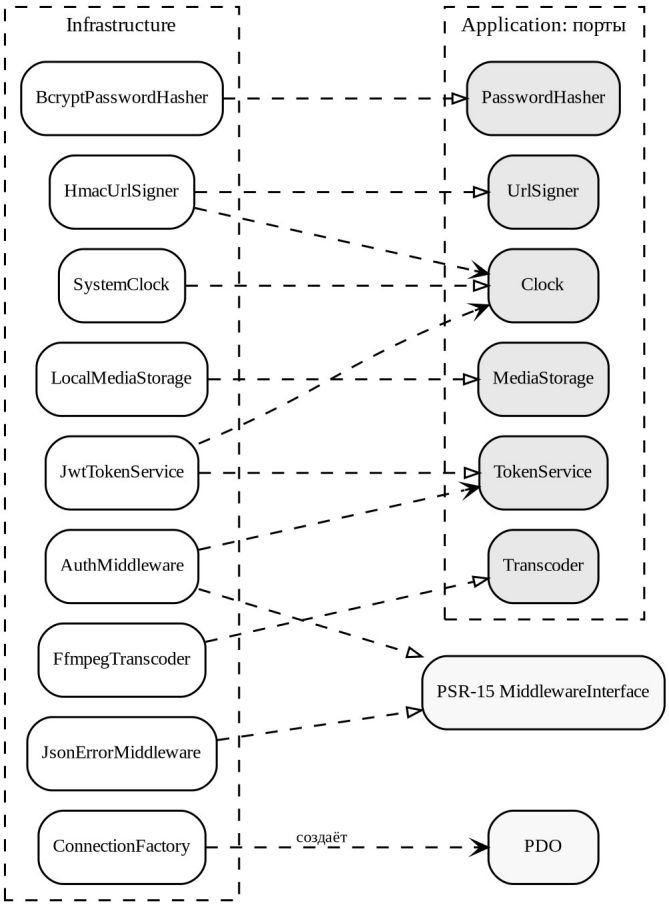


Рисунок 8 – Диаграмма классов слоя инфраструктуры (Infrastructure)

Связывание интерфейсов с реализациями выполняется в одном месте — файле config/container.php (корень композиции). Точки входа тоже относятся к внешнему слою: public/index.php создаёт приложение Slim и подключает маршруты (config/routes.php), bin/worker.php запускает фоновый обработчик транскодирования, а bin/create-admin.php создаёт учётную запись администратора. Обработчик запросов — веб-сервер nginx с PHP-FPM; схема развёртывания показана на рисунке 3. Внутренние слои не обращаются к контейнеру, поэтому не зависят от выбранных на этом уровне технологий.

3.6 Взаимодействие слоёв

Взаимодействие слоёв рассмотрим на двух ключевых сценариях. Первый — просмотр видео (рисунок 9). Запрос ссылки проходит через AuthMiddleware, который проверяет токен, затем контроллер передаёт идентификаторы сценарию RequestStream. Сценарий запрашивает контент и подписку через интерфейсы репозиториев, применяет правило доступа из сущности Content и получает от порта UrlSigner подпись. Клиент получает адрес плейлиста и запрашивает его уже без токена: StreamController проверяет

подпись, находит файл через порт MediaStorage и возвращает плейлист, в котором к каждому сегменту дописана подпись.

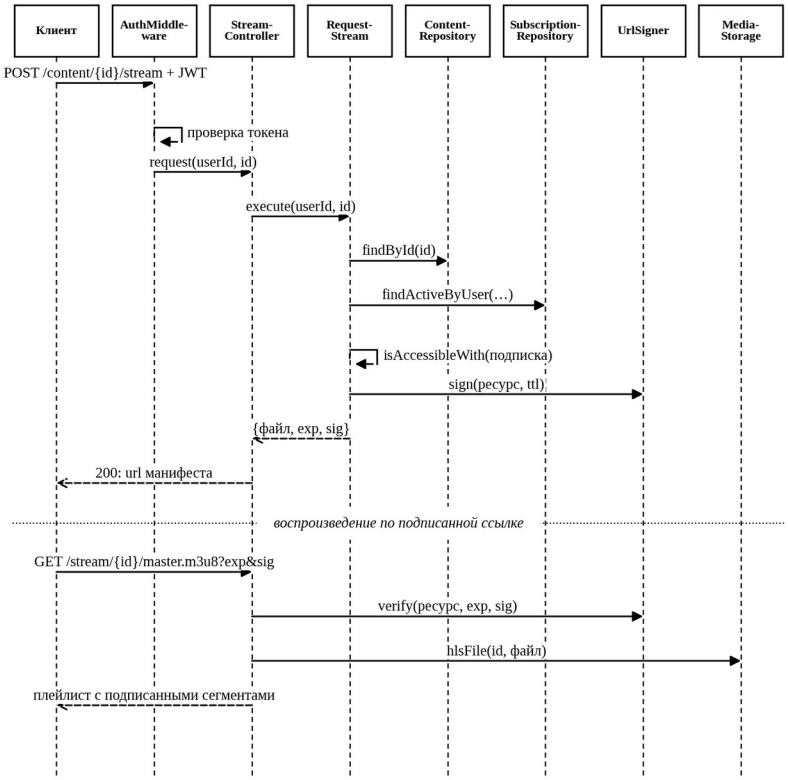


Рисунок 9 – Диаграмма последовательности сценария просмотра видео

Второй сценарий — загрузка видео и транскодирование (рисунок 10). В HTTP-запросе администратора выполняется только быстрая часть: сценарий AttachMedia проверяет существование контента и формат файла, сохраняет исходник, переводит контент в статус «обработка» и создаёт задачу в очереди. Ответ с кодом 202 возвращается сразу. Транскодирование выполняет отдельный процесс: сценарий ProcessNextTranscodeJob атомарно берёт задачу, запускает преобразование через порт Transcoder и после завершения переводит контент в статус «опубликован», а задачу — в «выполнена»; при ошибке контент получает статус «ошибка», а текст ошибки сохраняется в задаче.

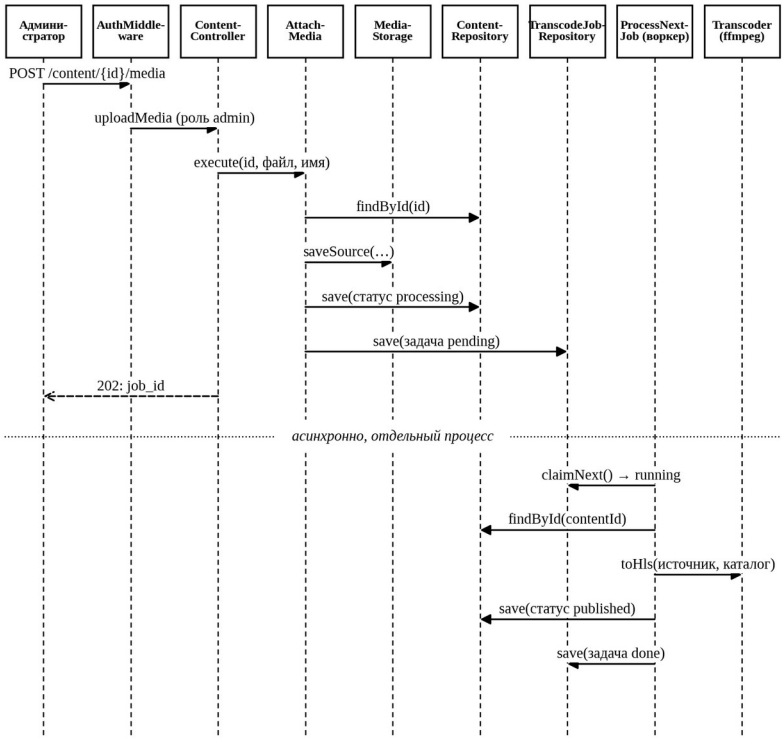


Рисунок 10 – Диаграмма последовательности загрузки и транскодирования видео

3.7 Соответствие архитектуры требованиям и допущенные компромиссы

Выбранная архитектура обеспечивает нефункциональные требования из таблицы 2. Сопровождаемость: бизнес-правила изолированы в слоях Domain и Application, а замена хранилища затрагивает только классы Pdo*Repository, транскодера или способа хранения файлов — только соответствующие классы слоя Infrastructure. Тестируемость: для проверки сценариев достаточно шести тестовых реализаций портов и репозиториев в памяти (каталог tests/Fake), без базы данных и веб-сервера. Производительность и надёжность: длительное транскодирование отделено от обработки запросов и выполняется отдельным процессом с хранением статусов в базе данных.

При реализации допущены следующие компромиссы, которые необходимо учитывать:

1. Исключения предметной области хранят рекомендуемый код HTTP-ответа. Более строгий вариант — отдельная таблица соответствия исключений кодам во внешнем слое.

2. Метод StreamController::serve содержит проверку подписи и чтение файла без отдельного сценария. Более строгий вариант — сценарий выдачи файла в слое Application.

3. Адаптеры хранилищ в слое Interfaces обращаются к классу PDO напрямую. PDO — часть языка PHP, а не внешний фреймворк, поэтому такая

зависимость допускалась; в образце из методических указаний [20] для этого используется абстрактный класс доступа к базе данных, который можно добавить при необходимости.

4. Задача транскодирования, взятая воркером, при его аварийной остановке остаётся в статусе «выполняется» (см. раздел 2.4).

3.8 Выводы по главе 3

1. На основе чистой архитектуры разработана архитектура системы из четырёх слоёв: Domain, Application, Interfaces и Infrastructure. Бизнес-правила размещены во внутренних слоях и не зависят от фреймворков и базы данных.

2. Состав слоёв описан диаграммами классов в нотации UML, построенными по исходному коду: 6 сущностей и 6 интерфейсов репозиториев, 10 сценариев и 6 портов, 4 контроллера и 6 PDO-адаптеров, 6 реализаций портов и 2 промежуточных слоя HTTP.

3. Соблюдение правила зависимостей проверяется тремя автоматическими архитектурными тестами; нарушений нет.

4. Отмечены четыре компромисса реализации; они не нарушают правила зависимостей и рассматриваются как направления улучшения. Результаты главы являются основой для главы 4 — описания разработки серверной части.

4 РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ИНТЕРНЕТ-РЕСУРСА

В настоящей главе описана реализация серверной части по архитектуре, разработанной в главе 3: структура проекта и запуск, база данных, ключевые механизмы (очередь транскодирования, выдача видео по подписанным ссылкам, подписки, защита), результаты тестирования, ограничения и направления развития. Исходный код приведён в проекте, в тексте даны только фрагменты, существенные для понимания решений.

4.1 Структура проекта и запуск

Проект организован по слоям архитектуры: каталог src содержит четыре подкаталога — Domain, Application, Interfaces и Infrastructure. Настройка зависимостей и маршруты вынесены в каталог config, точки входа — в public и bin, схема базы данных — в database, описание контейнеров — в docker и docker-compose.yml, тесты — в tests. Параметры (строка подключения к базе данных, секреты подписи токенов и ссылок, путь к хранилищу файлов, срок действия ссылки) задаются переменными окружения в файле .env.

Запуск выполняется командами Docker Compose: сборка и старт контейнеров (docker compose up -d --build), установка PHP-зависимостей (composer install в контейнере приложения), перезапуск воркера и создание администратора командой bin/create-admin.php. При первом старте контейнер PostgreSQL сам выполняет database/schema.sql. Подробная инструкция приведена в файле README проекта. Проверка выполнена в среде Windows с Docker Desktop (Docker 28.4.0, Docker Compose 2.39.2), PHP 8.3.35 и ffmpeg 7.1.5 в контейнере.

4.2 База данных

Схема данных состоит из шести таблиц (рисунок 11) и описана в database/schema.sql. Целостность данных обеспечивается ограничениями на уровне СУБД: внешними ключами, проверками допустимых значений и уникальностью (таблица 12). Роль пользователя и статусы контента и задач хранятся в строковых полях с ограничением CHECK. В таблицу тарифов при создании схемы добавляются два тарифа — Basic и Premium, каждый на 30 дней.

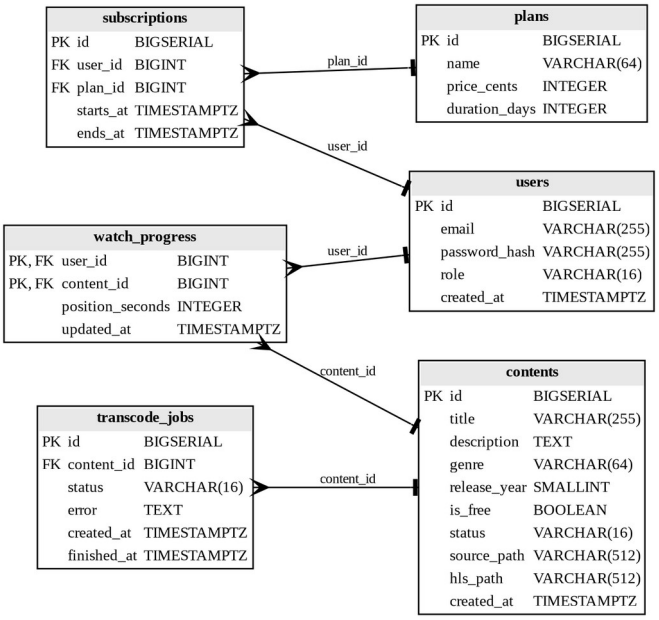


Рисунок 11 – Схема базы данных

Таблица 12 — Таблицы базы данных и ограничения целостности

Таблица	**Назначение**	**Ограничения и индексы**
users	Учётные записи	email уникален; role только viewer или admin
plans	Тарифы	name уникально; price_cents не меньше 0; duration_days больше 0
subscriptions	Подписки пользователей	Внешние ключи на users (с каскадным удалением) и plans; ends_at больше starts_at; индекс по (user_id, ends_at)
contents	Каталог контента	status только draft, processing, published или failed; индексы по genre и lower(title)
transcode_jobs	Очередь транскодирования	Внешний ключ на contents; status только pending, running, done или failed; индекс по (status, id)
watch_progress	Позиции просмотра	Составной первичный ключ (user_id, content_id); position_seconds не меньше 0

4.3 Реализация ключевых механизмов

4.3.1 Регистрация, вход и роли

Сценарий RegisterUser приводит адрес электронной почты к нижнему регистру, проверяет формат, длину пароля (не менее 8 символов) и уникальность, после чего сохраняет хеш bcrypt. Регистрация всегда создаёт пользователя с ролью «зритель»; администратор создаётся только командой bin/create-admin.php. Сценарий LoginUser возвращает одну и ту же ошибку и для неизвестного адреса, и для неверного пароля, чтобы нельзя было выяснить, зарегистрирован ли адрес. Выдаётся токен JWT со сроком действия

один час [19]. Промежуточный слой AuthMiddleware проверяет заголовок Authorization и роль: маршруты администратора вынесены в отдельную группу.

4.3.2 Загрузка видео и очередь транскодирования

Сценарий AttachMedia принимает файл с расширением mp4, mkv, mov, avi или webm, сохраняет его в каталог хранилища (sources/{id}), переводит контент в статус «обработка» и создаёт задачу в таблице transcode_jobs. HTTP-ответ возвращается сразу, не дожидаясь транскодирования. Лимит загрузки (2 ГБ) задан в конфигурации PHP-FPM и nginx.

Воркер (bin/worker.php) в бесконечном цикле вызывает сценарий ProcessNextTranscodeJob; если очередь пуста, он ждёт две секунды. Задача берётся одним запросом с блокировкой строки (листинг 1): конструкция SKIP LOCKED позволяет запустить несколько воркеров, не мешающих друг другу [31]. После выбора строки статус меняется на running в той же транзакции. При успехе контент получает статус published, задача — done; при исключении контент получает статус failed, а текст ошибки сохраняется в задаче.

```
SELECT id, content_id FROM transcode_jobs
WHERE status = 'pending'
ORDER BY id LIMIT 1
FOR UPDATE SKIP LOCKED;
```

Листинг 1 – Захват следующей задачи из очереди

4.3.3 Транскодирование в HLS

Транскодирование выполняет программа ffmpeg, которую запускает класс FfmpegTranscoder. Аргументы передаются массивом, без оболочки командной строки, поэтому имя файла не может внедрить команду. Параметры (листинг 2) подобраны так, чтобы результат воспроизводился большинством плееров: чётные размеры кадра и пиксельный формат yuv420p, профиль main кодека H.264, аудио AAC, ключевые кадры на границе каждого шестисекундного сегмента [33]. На выходе создаются плейлист master.m3u8 и сегменты seg_00000.ts, seg_00001.ts и далее.

```
-vf scale=trunc(iw/2)*2:trunc(ih/2)*2 -pix_fmt yuv420p
-c:v libx264 -preset veryfast -profile:v main
-force_key_frames expr:gte(t,n_forced*6)
-c:a aac -b:a 128k
-f hls -hls_time 6 -hls_playlist_type vod
-hls_segment_filename <каталог>/seg_%05d.ts <каталог>/master.m3u8
```

Листинг 2 – Основные параметры ffmpeg

4.3.4 Выдача видео по подписанным ссылкам

Выдача выполняется в два этапа. На первом авторизованный пользователь обращается к маршруту POST /content/{id}/stream: сценарий RequestStream проверяет, что контент опубликован, и правило доступа, после чего возвращает ссылку вида /stream/{id}/master.m3u8?exp=…&sig=…. На втором плеер запрашивает плейлист и сегменты по этой ссылке без токена. Подпись — код HMAC-SHA256 от строки «content:{id}|{exp}» с секретным ключом [36]; при проверке сначала сравнивается срок, затем подпись функцией hash_equals (листинг 3). Контроллер дописывает параметры exp и sig к адресам сегментов в плейлисте, а имя запрашиваемого файла проверяется регулярным выражением, допускающим только файлы .m3u8 и .ts. Плейлист отдаётся с заголовком Cache-Control: no-store, сегменты — с private, max-age=60.

```
public function sign(string $resource, int $ttlSeconds): array {
    $exp = $this->clock->now()->getTimestamp() + $ttlSeconds;
    return ['exp' => $exp, 'sig' => $this->mac($resource, $exp)];
}
public function verify(string $resource, int $exp, string $sig): bool {
    return $exp >= $this->clock->now()->getTimestamp()
        && hash_equals($this->mac($resource, $exp), $sig);
}
private function mac(string $resource, int $exp): string {
    return hash_hmac('sha256', $resource . '|' . $exp, $this->secret);
}
```

Листинг 3 – Подпись и проверка ссылок (HmacUrlSigner)

4.3.5 Подписки, позиция просмотра и история

Сценарий SubscribeToPlan создаёт подписку на срок, заданный тарифом. Если у пользователя уже есть активная подписка, новая начинается в момент окончания текущей, а не перекрывает её. Позиция просмотра сохраняется запросом INSERT … ON CONFLICT, то есть повторное сохранение обновляет запись, а не создаёт новую. История возвращает последние сохранённые позиции, по умолчанию не более 50 записей. Демонстрационная страница public/demo.html использует эти же маршруты и продолжает воспроизведение с сохранённой позиции.

4.3.6 Обработка ошибок и меры защиты

Единая обработка ошибок выполняется промежуточным слоем JsonErrorMiddleware: исключения предметной области преобразуются в ответ JSON с кодом из таблицы 9, а непредвиденные ошибки записываются в журнал, пользователю же возвращается общее сообщение без внутренних подробностей. Меры защиты сведены в таблицу 13 [17, 18].

Таблица 13 — Меры защиты

Угроза	Мера	Где реализована
Раскрытие паролей при утечке базы	Хеширование bcrypt	BcryptPasswordHasher
Выяснение зарегистрированных адресов	Одинаковое сообщение об ошибке входа	LoginUser
SQL-инъекции	Параметризованные запросы; отключена эмуляция подготовленных запросов; экранирование спецсимволов LIKE	Pdo*Repository, ConnectionFactory
Выход за пределы каталога	Белый список имён файлов	StreamController
Внедрение команд через имя файла	Запуск ffmpeg без оболочки, аргументы массивом	FfmpegTranscoder
Подделка и повторное использование ссылок	HMAC-SHA256, срок действия, сравнение через hash_equals	HmacUrlSigner
Нарушение авторизации	Роли в промежуточном слое; проверка права просмотра в сценарии	AuthMiddleware, RequestStream
Утечка внутренних сведений об ошибках	Общее сообщение при непредвиденной ошибке	JsonErrorMiddleware

4.4 Тестирование и результаты проверки

4.4.1 Автоматические тесты

Набор состоит из 15 тестов PHPUnit [38]: 12 unit-тестов сценариев (RegisterUser — 4, RequestStream — 5, SubscribeToPlan — 3) и 3 архитектурных теста правила зависимостей. Сценарии проверяются на шести тестовых реализациях в памяти (каталог tests/Fake). В контейнере приложения (PHP 8.3.35, PHPUnit 10.5.65) набор из 12 unit-тестов выполнен успешно: OK (12 tests, 16 assertions). Полный набор из 15 тестов выполнен успешно в среде разработки с PHP 8.3 и PHPUnit 9.6: OK (15 tests, 25 assertions).

4.4.2 Ручное тестирование

Работа системы проверена сценарием из HTTP-запросов (curl) и через демонстрационную страницу. Для проверки использовано тестовое видео 640×360, 20 секунд, созданное средствами ffmpeg. Для администратора страница содержит форму добавления видео: она создаёт карточку запросом POST /content, загружает файл запросом POST /content/{id}/media и ожидает появления ролика в каталоге. Результаты приведены в таблице 14.

Таблица 14 — Результаты ручного тестирования

№	Сценарий	Ожидаемый результат	Результат
1	Каталог при пустой базе	Пустой список	Совпало
2	Вход администратора	Выдан токен JWT	Совпало
3	Создание карточки контента	Карточка со статусом draft	Совпало
4	Загрузка видео	Задача создана (pending)	Совпало
5	Обработка воркером	В каталоге статус published	Совпало
6	Ссылка на бесплатный контент	Ссылка с параметрами exp и sig	Совпало
7	Плейлист по ссылке	4 сегмента (6, 6, 6, 2 с) с подписью	Совпало
8	Сегмент по ссылке	Код 200, video/mp2t	Совпало (137 428 байт)
9	Подделанная подпись	Отказ в доступе	Совпало
10	Запрос ссылки без токена	Требуется авторизация	Совпало
11	Платный контент без подписки	Требуется активная подписка	Совпало
12	Оформление подписки	Срок 30 дней	Совпало (с 04.10.2026 по 03.11.2026)
13	Платный контент с подпиской	Ссылка выдана	Совпало
14	Создание контента зрителем	Требуются права администратора	Совпало
15	Сохранение позиции и история	Позиция 42 с в истории	Совпало
16	Поиск по названию и жанру	Только подходящие записи	Совпало
17	Демонстрационная страница	Вход, каталог, воспроизведение	Совпало

Истечение срока подписи проверено отдельным скриптом при разработке: подпись со сроком в прошлом отклоняется. Сбой транскодирования (статус failed) и аварийная остановка воркера отдельными тестами не проверялись.

4.5 Ограничения и направления развития

Известные ограничения реализации и возможные способы их устранения сведены в таблицу 15.

Таблица 15 — Ограничения и направления развития

Ограничение	Последствие	Направление развития
Формируется один вариант качества	Нет адаптивного переключения битрейта	Несколько вариантов и главный плейлист HLS [5]

Ограничение	Последствие	Направление развития
Задача в статусе running не возвращается в очередь при аварии воркера	Контент остаётся в обработке	Тайм-аут и возврат «зависших» задач; брокер сообщений
Ссылка не привязана к пользователю; нет DRM	Ссылку можно передать до истечения срока	Привязка к пользователю, короткие сроки, DRM
Оплата не подключена	Подписка оформляется без оплаты	Платёжный шлюз
Нет ограничения числа попыток входа	Возможен подбор пароля	Лимит попыток и временная блокировка [18]
Передача по HTTP; пароль БД и секреты по умолчанию в файлах конфигурации	Риск при публикации сервиса	TLS на nginx; секреты вне репозитория
Поиск по подстроке без специального индекса	Замедление на большом каталоге	Триграммный индекс PostgreSQL
Файлы раздаёт PHP	Нагрузка на PHP-FPM	Раздача средствами nginx или CDN с проверкой подписи

4.6 Выводы по главе 4

1. Серверная часть реализована по четырёхслойной архитектуре; проект запускается командами Docker Compose, схема базы данных создаётся автоматически.

2. Реализованы все функциональные требования главы 1: регистрация и роли, каталог, загрузка видео, асинхронное транскодирование в HLS, выдача по подписанным ссылкам, подписки, позиция просмотра и история.

3. Работа системы подтверждена 15 автоматическими тестами и ручным сценарием из 17 проверок, результаты которого совпали с ожидаемыми.

4. Определены ограничения (один вариант качества, зависание задач при аварии воркера, отсутствие оплаты и DRM, нет ограничения попыток входа) и направления развития.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработана серверная часть системы стриминга контента, и все поставленные задачи решены.

В первой главе проанализирована предметная область, рассмотрены технологии адаптивной потоковой доставки по HTTP, сопоставлены четыре платформы-аналога, сформулированы функциональные и нефункциональные требования и построена матрица отслеживаемости, связывающая требования с реализацией и способами проверки.

Во второй главе обоснован выбор технологий: чистая архитектура, язык PHP 8.3, фреймворк Slim 4 с контейнером внедрения зависимостей PHP-DI, СУБД PostgreSQL 16 с очередью задач на основе блокировки строк, формат HLS, аутентификация по JWT, подписанные ссылки на видео и развёртывание в Docker.

В третьей главе спроектирована архитектура из четырёх слоёв — Domain, Application, Interfaces и Infrastructure. Слой сущностей содержит шесть сущностей и интерфейсы репозиториев, прикладной слой — десять сценариев использования и шесть портов, слой адаптеров — четыре контроллера и шесть репозиториев на PDO, слой инфраструктуры — реализации портов и промежуточные обработчики. Правило зависимостей проверяется автоматическим архитектурным тестом.

В четвёртой главе описана реализация: схема базы данных из шести таблиц, регистрация и вход, загрузка видео и очередь транскодирования, преобразование в HLS средствами FFmpeg, выдача сегментов по подписанным ссылкам с ограниченным сроком действия, подписки, сохранение позиции просмотра и история. Система запускается командой docker compose up, проверена 15 автоматическими тестами (25 проверок) и ручным прогоном полного сценария; для наглядной проверки добавлена демонстрационная страница на основе hls.js. Исходный код размещён в репозитории https://github.com/5hleev/streaming-backend.

Практическая значимость работы состоит в том, что полученное приложение можно использовать как основу небольшого сервиса видео по подписке, а принятая структура позволяет заменить СУБД, хранилище файлов или транскодер без изменения бизнес-логики.

Ограничения реализации указаны в разделе 4.5: единственный вариант качества, отсутствие автоматического возврата в очередь зависшей задачи, привязка ссылки не к пользователю, а к контенту, отсутствие оплаты, защиты от перебора паролей и шифрования соединения, а также раздача файлов средствами PHP. Направления развития: несколько вариантов качества с адаптивным переключением, выдача файлов через сеть доставки содержимого, интеграция с платёжным сервисом, ограничение частоты запросов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Тузовский, А. Ф. Проектирование и разработка web-приложений : учебное пособие для вузов / А. Ф. Тузовский. — Москва : Издательство Юрайт, 2021. — 218 с. — (Высшее образование). — ISBN 978-5-534-00515-8. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/469982 (дата обращения: 05.10.2026).

2. ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов. — Введ. 2016-06-01. — Москва, 2015.

3. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман ; пер. с англ. — Санкт-Петербург : Питер, 2018. — 640 с. — (Бестселлеры O’Reilly).

4. Sodagar, I. The MPEG-DASH Standard for Multimedia Streaming Over the Internet // IEEE MultiMedia. — 2011. — Vol. 18, no. 4. — P. 62–67. — DOI: 10.1109/MMUL.2011.71.

5. Pantos, R. HTTP Live Streaming : RFC 8216 / R. Pantos, W. May. — Internet Engineering Task Force (IETF), 2017. — Текст : электронный. — URL: https://www.rfc-editor.org/rfc/rfc8216 (дата обращения: 05.10.2026).

6. ISO/IEC 23009-1:2022. Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats. — Geneva : ISO, 2022.

7. A Survey on Bitrate Adaptation Schemes for Streaming Media Over HTTP / A. Bentaleb, B. Taani, A. C. Begen, C. Timmerer, R. Zimmermann // IEEE Communications Surveys & Tutorials. — 2019. — Vol. 21, no. 1. — P. 562–585. — DOI: 10.1109/COMST.2018.2862938.

8. YouTube recommended upload encoding settings // Справка YouTube [сайт]. — URL: https://support.google.com/youtube/answer/1722171 (дата обращения: 05.10.2026).

9. Encoding YouTube Videos at the Highest Quality // Videomaker [сайт]. — URL: https://www.videomaker.com/article/c05/17034-encoding-youtube-videos-at-the-highest-quality/ (дата обращения: 05.10.2026).

10. How to cancel your Twitch/YouTube/Mixer subscriptions // The Next Web [сайт]. — 20.02.2019. — URL: https://thenextweb.com/news/cancel-twitch-youtube-mixer-subscriptions (дата обращения: 05.10.2026).

11. Plans and Pricing // Netflix Help Center [сайт]. — URL: https://help.netflix.com/en/node/24926 (дата обращения: 05.10.2026).

12. Per-Title Encode Optimization // Netflix Technology Blog [сайт]. — 14.12.2015. — URL: https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2 (дата обращения: 05.10.2026).

13. Twitch State of Engineering 2023 // Twitch Blog [сайт]. — 28.09.2023. — URL: https://blog.twitch.tv/en/2023/09/28/twitch-state-of-engineering-2023 (дата обращения: 05.10.2026).

14. Subscriptions // Twitch Creator Camp [сайт]. — URL: https://www.twitch.tv/creatorcamp/en/level1/monetization/subscriptions/ (дата обращения: 05.10.2026).

15. Кинопоиск: фильмы и сериалы : мобильное приложение // App Store [сайт]. — URL: https://apps.apple.com/app/id477718890 (дата обращения: 05.10.2026).

16. ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering. — Geneva : ISO, 2018.

17. Хоффман, Э. Безопасность веб-приложений / Э. Хоффман. — Санкт-Петербург : Питер, 2021. — 336 с. — (Серия «Бестселлеры O’Reilly»).

18. OWASP API Security Top 10 — 2023 // OWASP Foundation [сайт]. — URL: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (дата обращения: 05.10.2026).

19. Jones, M. JSON Web Token (JWT) : RFC 7519 / M. Jones, J. Bradley, N. Sakimura. — Internet Engineering Task Force (IETF), 2015. — Текст : электронный. — URL: https://www.rfc-editor.org/rfc/rfc7519 (дата обращения: 05.10.2026).

20. Волков, М. Ю. Учебно-методическое пособие по выполнению курсовой работы по дисциплине «Разработка серверных частей интернет-ресурсов» для бакалавров, обучающихся по направлению подготовки 09.03.04 «Программная инженерия» очной формы обучения / М. Ю. Волков, В. В. Литвинов, А. А. Лобанов, А. В. Синицын. — Москва : МИРЭА — Российский технологический университет, 2021.

21. Fowler, M. Patterns of Enterprise Application Architecture / M. Fowler. — Boston : Addison-Wesley, 2002.

22. Эванс, Э. Предметно-ориентированное проектирование (DDD): структуризация сложных программных систем / Э. Эванс. — Москва : Вильямс, 2011. — 448 с.

23. Персиваль, Г. Паттерны разработки на Python: TDD, DDD и событийно-ориентированная архитектура / Г. Персиваль, Б. Грегори. — Санкт-Петербург : Питер, 2022. — 336 с.

24. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — Санкт-Петербург : Питер, 2021. — 352 с.

25. PHP: Руководство по PHP // PHP [сайт]. — URL: https://www.php.net/manual/ru/ (дата обращения: 05.10.2026).

26. Supported Versions // PHP [сайт]. — URL: https://www.php.net/supported-versions.php (дата обращения: 05.10.2026).

27. Slim 4 Documentation // Slim Framework [сайт]. — URL: https://www.slimframework.com/docs/v4/ (дата обращения: 05.10.2026).

28. PSR-7: HTTP message interfaces // PHP-FIG [сайт]. — URL: https://www.php-fig.org/psr/psr-7/ (дата обращения: 05.10.2026).

29. PSR-15: HTTP Server Request Handlers // PHP-FIG [сайт]. — URL: https://www.php-fig.org/psr/psr-15/ (дата обращения: 05.10.2026).

30. PostgreSQL 16 Documentation: INSERT // PostgreSQL Global Development Group [сайт]. — URL: https://www.postgresql.org/docs/16/sql-insert.html (дата обращения: 05.10.2026).

31. PostgreSQL 16 Documentation: SELECT // PostgreSQL Global Development Group [сайт]. — URL: https://www.postgresql.org/docs/16/sql-select.html (дата обращения: 05.10.2026).

32. hls.js : JavaScript HLS client using Media Source Extension // GitHub [сайт]. — URL: https://github.com/video-dev/hls.js (дата обращения: 05.10.2026).
33. FFmpeg Formats Documentation // FFmpeg [сайт]. — URL: https://ffmpeg.org/ffmpeg-formats.html (дата обращения: 05.10.2026).
34. password_hash // PHP: Руководство по PHP [сайт]. — URL: https://www.php.net/manual/ru/function.password-hash.php (дата обращения: 05.10.2026).
35. Password Storage Cheat Sheet // OWASP Cheat Sheet Series [сайт]. — URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 05.10.2026).
36. Krawczyk, H. HMAC: Keyed-Hashing for Message Authentication : RFC 2104 / H. Krawczyk, M. Bellare, R. Canetti. — Internet Engineering Task Force (IETF), 1997. — Текст : электронный. — URL: https://www.rfc-editor.org/rfc/rfc2104 (дата обращения: 05.10.2026).
37. Docker Compose overview // Docker Docs [сайт]. — URL: https://docs.docker.com/compose/ (дата обращения: 05.10.2026).
38. PHPUnit documentation // PHPUnit [сайт]. — URL: https://phpunit.de/documentation.html (дата обращения: 05.10.2026).
39. Fowler, M. UML Distilled: A Brief Guide to the Standard Object Modeling Language / M. Fowler. — 3rd ed. — Boston : Addison-Wesley Professional, 2003. — 208 p. — ISBN 978-0-321-19368-1.
40. ISO/IEC 19505-2:2012. Information technology — Object Management Group Unified Modeling Language (OMG UML) — Part 2: Superstructure. — Geneva : ISO, 2012.

ПРИЛОЖЕНИЕ А

(обязательное)

Схема базы данных

Файл database/schema.sql

```
CREATE TABLE users (
    id            BIGSERIAL PRIMARY KEY,
    email         VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role          VARCHAR(16)  NOT NULL DEFAULT 'viewer' CHECK (role IN ('viewer', 'admin')),
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);


CREATE TABLE plans (
    id            BIGSERIAL PRIMARY KEY,
    name          VARCHAR(64) NOT NULL UNIQUE,
    price_cents   INTEGER     NOT NULL CHECK (price_cents >= 0),
    duration_days INTEGER     NOT NULL CHECK (duration_days > 0)
);


CREATE TABLE subscriptions (
    id        BIGSERIAL PRIMARY KEY,
    user_id   BIGINT      NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    plan_id   BIGINT      NOT NULL REFERENCES plans(id),
    starts_at TIMESTAMPTZ NOT NULL,
    ends_at   TIMESTAMPTZ NOT NULL,
    CHECK (ends_at > starts_at)
);
CREATE INDEX idx_subscriptions_user ON subscriptions(user_id, ends_at);


CREATE TABLE contents (
    id            BIGSERIAL PRIMARY KEY,
    title         VARCHAR(255) NOT NULL,
    description   TEXT         NOT NULL DEFAULT '',
    genre         VARCHAR(64)  NOT NULL,
    release_year  SMALLINT     NOT NULL,
    is_free       BOOLEAN      NOT NULL DEFAULT FALSE,
    status        VARCHAR(16)  NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'processing',
'published', 'failed')),
    source_path   VARCHAR(512),
    hls_path      VARCHAR(512),
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE INDEX idx_contents_genre ON contents(genre);
CREATE INDEX idx_contents_title ON contents(lower(title));


CREATE TABLE transcode_jobs (
    id          BIGSERIAL PRIMARY KEY,
    content_id  BIGINT      NOT NULL REFERENCES contents(id) ON DELETE CASCADE,
    status      VARCHAR(16) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'running', 'done',
'failed')),
    error       TEXT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    finished_at TIMESTAMPTZ
);
CREATE INDEX idx_jobs_status ON transcode_jobs(status, id);


CREATE TABLE watch_progress (
    user_id          BIGINT      NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    content_id       BIGINT      NOT NULL REFERENCES contents(id) ON DELETE CASCADE,
    position_seconds INTEGER     NOT NULL CHECK (position_seconds >= 0),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, content_id)
);


INSERT INTO plans (name, price_cents, duration_days) VALUES
    ('Basic', 29900, 30),
    ('Premium', 49900, 30);
```